DEEP LEARNING / COURSE PROJECT

LeafLens

Explainable crop leaf disease classification

A closer look. A healthier harvest.

Atul Biju · ASI23CA023

Athul M B · ASI23CA022

Athul Chandran · ASI23CA021

CNN baseline · VGG16 transfer learning · Grad-CAM

leaflens-black.vercel.app

11 October 2026

LeafLens: Explainable Crop Leaf Disease Classification

Deep Learning course project

Team member	Registration number
Atul Biju	ASI23CA023
Athul M B	ASI23CA022
Athul Chandran	ASI23CA021

Abstract

LeafLens investigates image-based crop leaf disease classification using a custom convolutional neural network and transfer learning with VGG16. The system integrates a reproducible training pipeline, held-out evaluation, Grad-CAM visual explanations and a responsive web interface. It focuses on the tomato, potato and bell pepper classes represented by the assigned PlantVillage Kaggle dataset. A prediction includes ranked classes, an uncalibrated confidence score and an attention map. The interface presents uncertainty and limitations alongside results. The intended social contribution is a low-cost observation aid; field deployment effectiveness is not established by a controlled-background dataset.

1. Problem and motivation

Crop health observations can be expensive or difficult to obtain frequently. Leaf photographs are inexpensive, but symptom interpretation needs expertise. The project asks whether convolutional visual features can distinguish the dataset's disease classes and whether transfer learning improves performance over a CNN trained from scratch. The interface helps users inspect results without concealing the limits of the experiment.

Research questions:

1. How does a small custom CNN compare with VGG16 transfer learning under the same image split?

2. Does fine-tuning VGG16's final convolutional block improve validation performance beyond training its classifier alone?
3. Which classes are confused most often, and how does confidence relate to correctness?
4. Which image regions influence the predicted class according to Grad-CAM?

2. Dataset collection and preparation — Week 1

The assigned source is PlantVillage on Kaggle. The downloader discovers the actual class directory rather than assuming an archive layout. An alternative script retrieves a deterministic, explicitly labeled subset from the original PlantVillage authors' repository, recording its exact source commit and selected files. This alternative is not described as a completed Kaggle download.

The selected scope is 15 classes across tomato, potato and bell pepper, including healthy leaves. This is the scope of this dataset variant, not every class in all PlantVillage releases. Inspect the actual manifest for class names and counts.

Preparation removes identical decoded RGB pixels before splitting, so exact copies cannot appear on both sides of evaluation. A cross-label duplicate is treated as an annotation conflict and rejected. A seeded stratified 70/15/15 train/validation/test partition ensures class coverage. Near duplicates and multiple photographs of the same physical leaf are **not** verified by pixel hashing; therefore, the resulting test set is not asserted to be fully leaf-independent.

Images are resized to the configured resolution and normalized using ImageNet mean `(0.485, 0.456, 0.406)` and standard deviation `(0.229, 0.224, 0.225)`. Training augmentation includes horizontal/vertical flips, rotations up to 20 degrees, and mild brightness, contrast and saturation variation. Validation and test data receive deterministic transforms only. No augmentation is performed before the split.

3. Architecture setup and base model training — Week 1

Custom CNN baseline

Four blocks contain a 3×3 convolution, batch normalization, ReLU and 2×2 max pooling. Channel widths are 32, 64, 128 and 256. Global average pooling produces a compact feature vector. Dropout (0.3) and a final linear layer produce one logit per class. Its weights begin randomly initialized, providing a from-scratch comparison.

VGG16 transfer learning

VGG16's ImageNet convolutional backbone is retained. The original large fully connected classifier is replaced with global average pooling and a `512 → 256 → number_of_classes` head, with ReLU and dropout (0.35). This substantially reduces classifier memory and parameter count while preserving the VGG16 feature extractor. It is a VGG16-based transfer learning model, not the unchanged original ImageNet architecture.

During the first stage, convolutional parameters are frozen and only the disease head is optimized with AdamW at learning rate `1e-3`. Class weights, computed from training counts only, address imbalance; cross-entropy uses label smoothing `0.05`. Weight decay is `1e-4`.

4. Fine-tuning — Week 2

The final VGG16 convolutional block is unfrozen while earlier layers remain frozen. AdamW is reset with a lower learning rate `1e-5`, reducing the risk of overwriting pretrained representations. A validation-accuracy scheduler reduces the learning rate on a plateau. Early stopping is allowed only after the fine-tuning phase has run, so a frozen-stage plateau cannot skip the required second stage. The best checkpoint is selected by validation accuracy across both stages. If the frozen-stage checkpoint remains best, that fact is preserved rather than hidden.

CUDA runs use mixed precision via `torch.amp` to reduce memory use. CPU execution remains supported. The default 224×224 resolution and batch size 16 target modest hardware; a smaller measured demonstration run may use a documented resolution and subset. Both architectures must use the same manifest fingerprint for comparison.

5. Evaluation and visualization — Week 2

The test partition is used after training, never for checkpoint selection. Report accuracy, macro F1 (each class has equal weight), weighted F1, per-class precision/recall/F1, support and a confusion matrix. Learning curves show training and validation accuracy/loss. A ten-bin expected calibration error (ECE) audits the difference between model confidence and empirical correctness. It does not calibrate the model.

Measured experiment snapshots live in `docs/results/` after runs are completed. The live Model lab reads saved experiment artifacts. Never quote an accuracy unless a corresponding completed test result exists. Test-fixture checkpoints exercise software plumbing and are not evidence of classification quality.

Completed local demonstration

Model	Held-out test accuracy	Macro F1	Best checkpoint
Custom CNN	77.78%	76.56%	Epoch 9, baseline
VGG16 transfer learning	70.22%	70.17%	Epoch 10, fine-tuning

These are measured results from **1,500 authentic upstream PlantVillage images**, capped at 100 images per class, with 128×128 inputs, seed 42 and shared 1,050/225/225 train/validation/test partitions. The full Kaggle archive download was bandwidth-limited. In this setting the CNN performed better; pretrained features are not guaranteed to outperform a task-specific baseline under every dataset size, resolution and optimization budget. This single run does not establish statistical significance. Larger full-data experiments, repeated seeds and external validation are needed before broader conclusions. See the complete experiment record.

Grad-CAM uses the gradient of the predicted class with respect to the final convolutional activations. Spatially averaged gradients weight feature maps; the weighted sum is rectified, normalized and resized. The overlay illustrates class-specific sensitivity. It is **not** a segmentation mask, severity estimate, causal explanation or proof that the model used the right biological feature. Grad-CAM also works when the backbone is frozen by explicitly enabling input gradients.

6. System and UI

React/Vite provides four views: Diagnose, Disease atlas, Model lab and Field notes. FastAPI validates uploaded JPEG/PNG/WebP photos (4 MB hosted / 10 MB local maximum, 20 megapixels maximum), loads checkpoints and produces ranked predictions with attention overlays. Image orientation is corrected before inference. Model inference is serialized to avoid overlapping gradient hooks. A missing or incompatible checkpoint yields a recoverable error rather than an invented result.

The field-station design uses forest green, pale botanical accents, deliberate typography and a photographic leaf panel. The interface supports keyboard navigation, dialog focus trapping, reduced motion and mobile layouts. Field notes store up to 20 small thumbnails and results in local browser storage; uploaded originals are not persisted by the API. No external image-analysis API is used. Inference runs in the application's Python service, locally or on Vercel.

7. Social relevance and limitations

The system could help organize low-cost crop monitoring observations and improve information brought to agricultural experts. Demonstrated software functionality does not establish that it improves farmer outcomes. PlantVillage's relatively controlled photographs differ from field backgrounds, illumination, occlusion, growth stages and devices. Unknown diseases, non-leaf inputs and nutritional deficiencies may receive confident but incorrect known-class predictions. Softmax does not provide validated out-of-distribution rejection. The UI's 0.70 review threshold is a transparent heuristic, not a clinically or agronomically validated cutoff.

No treatment recommendation, pesticide dosage, yield improvement or field accuracy is claimed. External dataset validation, leaf-level grouping, field photographs, probability calibration, and user studies are future work.

8. Course activity mapping

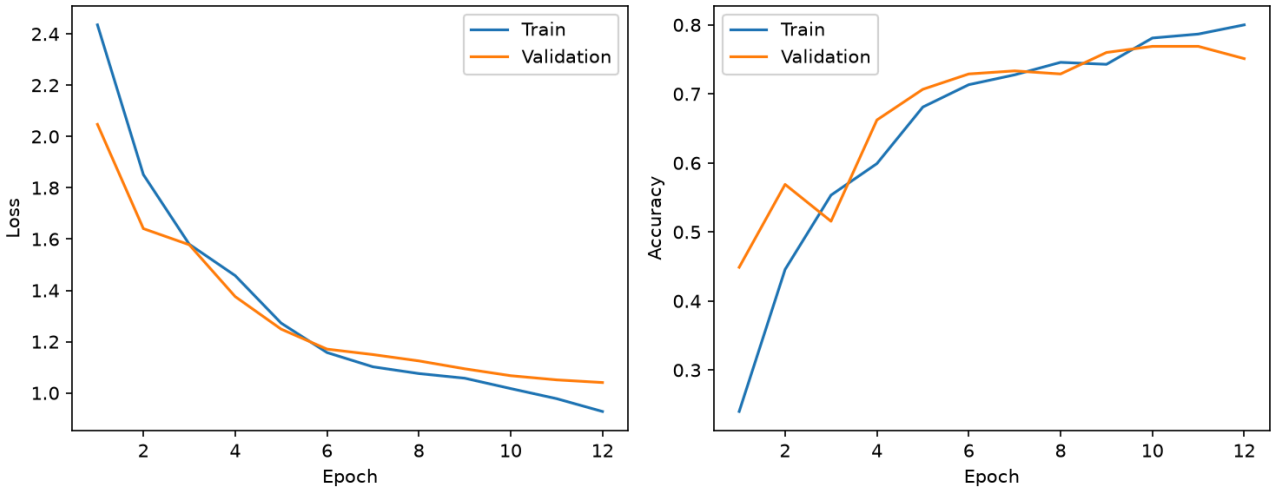
Professor's activity	Implementation / evidence
Dataset collection	Kaggle downloader; documented upstream subset alternative; source provenance
Preprocessing	Duplicate removal, split manifest, augmentation, normalization
Architecture setup	`ml/models.py`: CNN and VGG16 with compact head
Base training	`ml/train.py`: CNN and frozen VGG16 phase
Fine-tuning	VGG16 final block, lower learning rate, early stopping
Evaluation	Accuracy/F1, per-class metrics, confusion matrix, ECE
Visualization	Learning curves, heatmaps, interactive Model lab
Presentation/report	This report, `docs/PRESENTATION.html`, notebook, viva notes

References

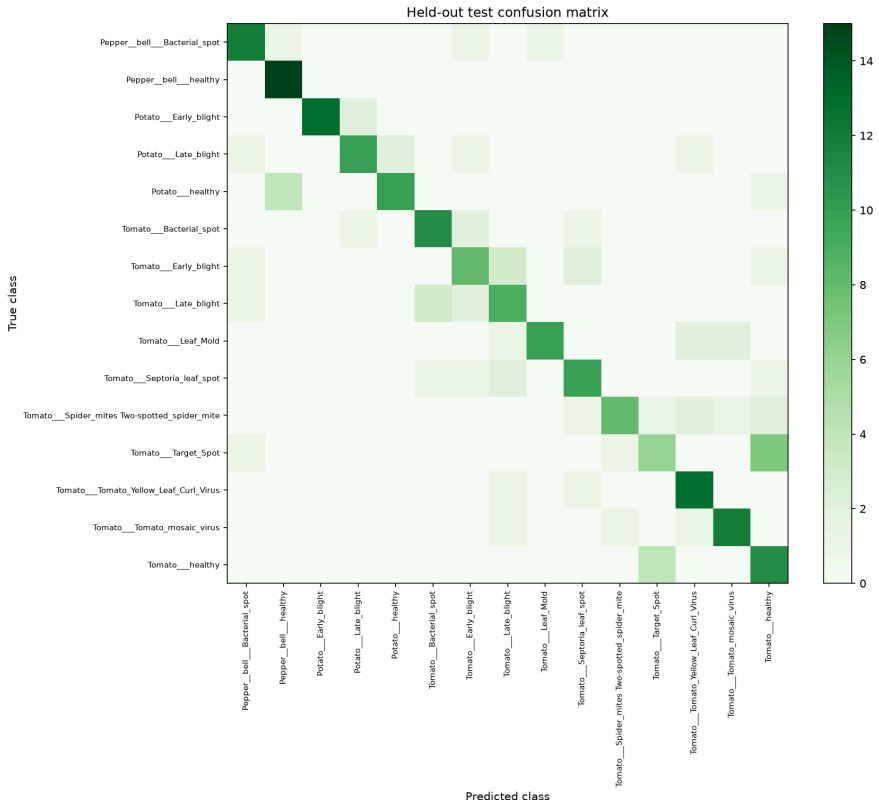
1. PlantVillage Kaggle dataset.
2. Original PlantVillage dataset repository and provenance.

3. Simonyan & Zisserman, Very Deep Convolutional Networks for Large-Scale Image Recognition, 2014.
4. Selvaraju et al., Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization, 2016.
5. Torchvision VGG16 documentation.
6. PyTorch automatic mixed precision.

Measured experiment visualizations



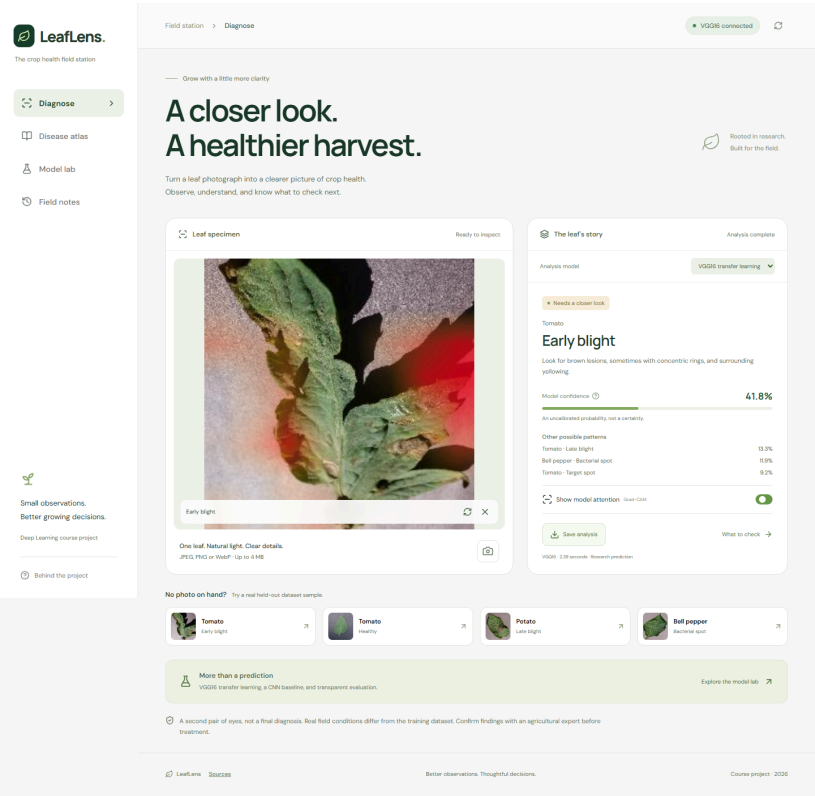
VGG16 training and validation curves.



VGG16 held-out confusion matrix; 225 test images.

Deployed application

The complete application hosts both trained models on Vercel. Original uploaded photos are not retained by the service; compact field notes remain in the user’s browser.



Live diagnosis interface with model prediction and Grad-CAM controls.

Live application: https://leaflens-black.vercel.app

Verification: 20 Python tests, 7 browser checks, and GitHub CI passed. These checks establish software behavior, not field classification accuracy.